EN3150 Assignment 03

Resource-Constrained CNN: Task Checklist

Shared checklist for the group; no member assignments.

Scope. Build two custom CNNs and fine-tune two pretrained lightweight models on one low-resolution dataset. Compare accuracy, memory and computational cost. The assignment brief is the source of requirements; workflow choices below make the experiments reproducible.

1. Project decisions and setup

- ☐ Confirm the dataset (EuroSAT RGB is a candidate), framework, class order, and input resolution (at most 64×64). Do not use CIFAR-10.
- ☐ Create a Git repository with a README, dependency file, configuration, and runnable instructions. Commit throughout the project.
- ☐ Set a random seed and shared rules for data splits, metrics, checkpoint names, and result files.
- ☐ Decide what is measured as model size and computational cost; record hardware/software used for timings.

2. Dataset and data pipeline

- ☐ Download the selected dataset from its official source. For EuroSAT, inspect `https://github.com/phelber/eurosat` and use the RGB image release.
- ☐ Verify image format and count, labels, class distribution, and sample images; document exclusions or corrupt files.
- ☐ Resize or confirm all images are at most 64×64 pixels.
- ☐ Create and save a reproducible 70% train / 15% validation / 15% test split; stratify where practical. Check for overlap.
- ☐ Implement a shared data loader and consistent preprocessing. Apply augmentation only to training data, if used.
- ☐ Record dataset details in the sheet referenced by the assignment brief.

3. Read and position relevant work

- ☐ Read Helber et al., *EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification* (dataset and baseline methods).
- ☐ Read Krizhevsky et al., *ImageNet Classification with Deep Convolutional Neural Networks* (standard convolutions and max-pooling); do not present it as the same-sized architecture as Model A.
- ☐ Read Howard et al., *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications* (depthwise-separable convolution and efficiency); Model B is a custom model, not a reproduction of MobileNet.
- ☐ If using global average pooling, read Lin et al., *Network In Network*. Optionally consult Chollet, *Xception*, for depthwise-separable convolution.
- ☐ Record proper citations in the report and distinguish published results from your own results.

4. Design the two custom architectures

- ☐ Implement Model A with standard 2D convolutions interleaved with max-pooling.
- ☐ Implement Model B by using depthwise-separable convolutions; ensure **no more than 100,000 trainable parameters**.
- ☐ Document every layer: input/output shape, kernel, channels, activation, pooling, and classifier units.
- ☐ Calculate trainable parameters layer by layer for both models and verify against framework summaries.
- ☐ Justify activations for low-resource hardware. Estimate MACs/FLOPs using one stated convention, if feasible.

5. Shared training and evaluation code

- ☐ Build a configurable training script with saved seed/configuration, checkpoints, per-epoch times, and training/validation histories.
- ☐ Build one test evaluator for accuracy, confusion matrix, precision, and recall. State whether metrics are per-class, macro, or both.
- ☐ Write metrics to JSON/CSV and save plots with labeled axes; do not manually transcribe results.
- ☐ Keep the test set out of optimizer tuning, checkpoint selection, and model selection.

6. Train and compare custom models

- ☐ Choose a primary optimizer and learning rate; explain the choice.
- ☐ Compare the primary optimizer with plain SGD and SGD with momentum under documented settings. Explain momentum's effect on convergence/performance.
- ☐ Train **both Model A and Model B for at least 20 epochs**. Save training and validation loss curves.
- ☐ Evaluate both on the held-out test set: accuracy, confusion matrix, precision, and recall.
- ☐ Create the required A-versus-B table: parameter count, estimated disk size (KB), time per epoch, and test accuracy.
- ☐ Discuss the observed trade-offs of standard versus depthwise-separable convolution.

7. Fine-tune two lightweight pretrained networks

- ☐ Select two eligible pretrained architectures (e.g., MobileNetV2 and EfficientNet-B0) and adapt their classifier heads.
- ☐ Fine-tune both on the same saved train/validation/test split. Document preprocessing and fine-tuning settings.
- ☐ Evaluate both using the shared test evaluator. Report test metrics, total parameter counts, and model sizes (MB).

8. Final analysis and report

- ☐ Compare Model B with both pretrained networks on accuracy, memory footprint, and computational cost; define all measurement methods and units.
- ☐ Discuss advantages, limitations, and observed accuracy–memory–compute trade-offs. Do not claim a model is faster without a measured or justified cost comparison.
- ☐ Write methods, dataset, architectures, parameter calculations, optimizer analysis, results, limitations, and conclusions. Include citations, figures, and comparison tables.
- ☐ Check every report figure and number against saved results. Include all group members' names and index numbers and a GitHub/SVN profile link.

9. Verification and submission

- ☐ Run the submitted code from a clean environment using the README; check that saved split and metrics can be reproduced.
- ☐ Proofread the PDF report, including legends, units, equations, citations, and interpretation of results.
- ☐ Submit the report PDF and runnable, documented code **separately** to Moodle. Use the required base name `Yourgroupno_A03_EN3150`. One submission per group.

Optional only after required work

- ☐ Knowledge distillation, INT8 quantization, architecture search, or device benchmarks. These are **not required**.

Suggested project structure

Independent runs, shared infrastructure. Each model has its own implementation and configuration. The data loader, training loop, evaluator, and split manifests are shared.

```
EN3150_A03/
  README.md
  pyproject.toml
  .gitignore
  configs/
    base.yaml
    models/
      model_a.yaml
      model_b.yaml
      mobilenet_v2.yaml
      efficientnet_b0.yaml
    optimizers/
      adam.yaml
      sgd.yaml
      sgd_momentum.yaml
  data/
    README.md
    raw/                    [ignored]
    splits/
      train.csv
      val.csv
      test.csv
  src/en3150_a03/
    __init__.py
    data.py
    models/
      __init__.py
      model_a.py
      model_b.py
      mobilenet_v2.py
      efficientnet_b0.py
    train.py
    evaluate.py
    metrics.py
    measure.py
    config.py
```

```
EN3150_A03/ (continued)
  scripts/
    prepare_data.py
    train_model.py
    evaluate_model.py
    compare_models.py
    make_figures.py
  tests/
    test_splits.py
    test_model_shapes.py
    test_model_b_params.py
    test_metrics.py
  results/
    model_a/
      adam/
      sgd/
      sgd_momentum/
    model_b/
      adam/
      sgd/
      sgd_momentum/
    mobilenet_v2/
    efficientnet_b0/
    comparisons/
  report/
    report.tex
    references.bib
    figures/
    tables/
  docs/
    task_checklist.tex
    task_checklist.pdf
    experiment_notes.md
```

Rule: Train each model separately and save its checkpoints, histories, timings, and metrics in its own results folder. All four models read the same split files and use the same evaluator. Track source code and small split manifests; do not commit raw images or large checkpoints.